2-HOUR BUILD ASSIGNMENT

Build Your Own Engineering Degree OS

A self-managed 1st-to-8th semester engineering degree command center with schedules, syllabus, assessments, progress, scorecard, plan/checklist, and an AI chatbot.

⏱️	👥	💻	🚀
120 min	**2-4**	**Free**	**1**
Build time	Students / team	Core tool cost	Working app

> **Mission**
> By the end of 120 minutes, your team must have a working Streamlit web app that tracks an engineering degree from Semester 1 to Semester 8, can be maintained through simple CSV data, answers student questions through a chatbot, and is deployed to a shareable URL.

Suggested app name: MY ENGINEERING DEGREE OS

You are building a useful student product - not a perfect university ERP.

1. What you are building

> **Product concept**
> One personal dashboard that lets an engineering student see the complete 8-semester journey, know what is coming next, record academic progress, and ask questions without searching through multiple PDFs, calendars and spreadsheets.

Required tab / page	Minimum content	Pass condition
Dashboard	Degree overview, current semester, completion %, upcoming assessment, next holiday, current CGPA/score summary.	Visible and usable in the running app
Semesters 1-8	Eight semester cards with course count, credits, completion status and semester progress.	Visible and usable in the running app
Courses & Syllabus	Course code, course title, credits, faculty/owner, status and short syllabus/topic list.	Visible and usable in the running app
University Syllabus	Overall university / programme syllabus summary and source/reference field.	Visible and usable in the running app
Academic / Leave Calendar	Academic dates and leave/holiday calendar, filterable by academic year and semester.	Visible and usable in the running app
Internal Assessments	IA1, IA2, lab tests, assignments, project reviews and exam dates.	Visible and usable in the running app
Status Tracker	Not started / In progress / Completed status by course and semester.	Visible and usable in the running app
Scorecard	Marks, grade, credits, semester average and cumulative progress.	Visible and usable in the running app
Plan & Checklist	A visible checklist for degree milestones, exam preparation, documents, projects and placement readiness.	Visible and usable in the running app
Ask Degree AI	Chatbot for questions such as “What subjects are left in Sem 5?” or “When is my next IA?”	Visible and usable in the running app
Data Admin	Simple editable data tables so the app can be maintained without rewriting code.	Visible and usable in the running app

2. Free tool stack

Use the simplest stack that can be completed in class:

Tool	Purpose	Cost for assignment	Link
Python 3 + Streamlit	Build the web app locally.	Free / open source	Open
CSV files + Pandas	Store semester, course, calendar, assessment and score data.	Free	Open
GitHub Free	Save the project and make it deployable.	Free plan	Open
Streamlit Community Cloud	Deploy the app to a shareable URL.	Free for community / educational apps	Open
Google AI Studio + Gemini API	Optional AI layer for the chatbot.	Free tier available	Open
VS Code or GitHub Codespaces	Edit code locally or in the browser.	Free option	Open

> **Important**
> For this 2-hour build, do not create login/SSO, payment, attendance biometrics, or a full university ERP. Those are future enhancements. The goal is a complete student-facing degree tracker that works end-to-end.

3. The 120-minute clock

Time	Work	Do not move on until...
0-10 min	Set up tools and project folder	The phase output is visible or saved
10-25 min	Create the data model	The phase output is visible or saved
25-45 min	Build navigation + dashboard	The phase output is visible or saved
45-70 min	Build academic tabs	The phase output is visible or saved
70-85 min	Build status, scorecard, plan/checklist	The phase output is visible or saved
85-105 min	Add chatbot	The phase output is visible or saved
105-112 min	Test and fix	The phase output is visible or saved
112-120 min	Deploy and submit	The phase output is visible or saved

Phase 1 - Set up the project | 0-10 min

> **Output for this phase**
> A folder/repository with app.py, requirements.txt and a data folder.

1. Create a folder named engineering-degree-os.
2. Inside it create: app.py, requirements.txt, and a folder named data.
3. Put this in requirements.txt: streamlit, pandas, google-genai.
4. Open Terminal / Command Prompt in the folder and run: pip install streamlit pandas google-genai.
5. Run: streamlit hello. If the Streamlit demo opens, your environment is ready.
6. Create a free GitHub repository named engineering-degree-os now so deployment later takes less than 5 minutes.

> **Browser-only alternative**
> If Python is not installed, use GitHub Codespaces from the repository and run the same commands in the browser. Streamlit documents Community Cloud + Codespaces as a free cloud-development option.

Phase 2 - Create the degree data | 10-25 min

> **Output for this phase**
> Five small CSV files representing the whole 1st-to-8th semester journey.

1. In data/ create semesters.csv, courses.csv, academic_calendar.csv, assessments.csv, and scores.csv.
2. Populate semesters.csv with Semester 1 through Semester 8 and a status column.
3. Add at least 3 sample courses per semester to courses.csv. You do not need the full university syllabus during class; use representative sample rows and prove the design works for all 8 semesters.
4. Add holidays / leave dates and semester events to academic_calendar.csv. Include academic_year and semester so the app can filter them.
5. Add IA1, IA2, assignment, lab evaluation, project review, and end-semester exam examples to assessments.csv.
6. Add sample marks/grades to scores.csv for at least two semesters so the Scorecard has something to calculate.

Minimum CSV schemas

```
semesters.csv
semester,status,total_credits,completed_credits
1,Completed,22,22
2,Completed,23,23
3,In Progress,24,12
4,Not Started,24,0
... continue through 8

courses.csv
semester,course_code,course_name,credits,syllabus,status
1,MA101,Engineering Mathematics I,4,"Matrices; calculus; differential equations",Completed
1,PH101,Engineering Physics,4,"Waves; optics; semiconductors",Completed
3,CS301,Data Structures,4,"Stacks; queues; trees; graphs",In Progress

academic_calendar.csv
```

```
academic_year,semester,date,event_type,title
2026-27,3,2026-11-14,Holiday,Children's Day / College Holiday
2026-27,3,2026-11-20,Academic,Last working day

assessments.csv
semester,course_code,assessment_type,date,max_marks,status
3,CS301,IA1,2026-11-10,30,Planned
3,CS301,Assignment,2026-11-18,10,Planned

scores.csv
semester,course_code,assessment_type,score,max_marks,grade
1,MA101,End Semester,82,100,A
1,PH101,End Semester,76,100,B+
```

> **Engineering habit**
> Data first, UI second. If the data is clear and consistent, the screens, scorecard and chatbot become much easier to build.

Phase 3 - Build the navigation and dashboard | 25-45 min

> **Output for this phase**
> A running Streamlit app with all required navigation items visible and a useful home dashboard.

1. Open Google AI Studio (or your preferred free coding assistant) and paste the Master Build Prompt on the next page.
2. Ask it to generate a single-file app.py first. Do not request a complex database or authentication.
3. Paste the generated app.py into your project and run: streamlit run app.py.
4. Confirm the left navigation visibly contains Dashboard, Semesters, Courses & Syllabus, University Syllabus, Academic Calendar, Assessments, Status Tracker, Scorecard, Plan & Checklist, Ask Degree AI, and Data Admin.
5. On Dashboard show: current semester, degree completion %, completed credits, next assessment, next leave/holiday, and a simple semester-progress chart.
6. Fix only blocking errors. Do not spend time changing fonts, logos or animations.

Master Build Prompt - paste this into Google AI Studio

```
You are an expert Python + Streamlit engineer. Build a clean, student-friendly app named "My Engineering Degree OS" for an engineering student to manage Semester 1 through Semester 8.

TECHNICAL CONSTRAINTS
- Use Python, Streamlit and Pandas only for the core app.
- Store maintainable data in CSV files under a /data folder.
- Create one app.py file for this class assignment.
- If a CSV file is missing, show a friendly warning instead of crashing.
- Keep the UI clean, responsive and easy for a student to understand.

TOP-LEVEL NAVIGATION
1. Dashboard
2. Semesters 1-8
3. Courses & Syllabus
```

```
4. University Syllabus
5. Academic / Leave Calendar
6. Internal Assessments
7. Status Tracker
8. Scorecard
9. Plan & Checklist
10. Ask Degree AI
11. Data Admin

FUNCTIONS
- Dashboard: current semester, degree completion %, completed credits, next assessment, next holiday/leave, score summary, semester progress.
- Semesters: cards for all 8 semesters and filterable semester details.
- Courses: table filtered by semester; show course code, title, credits, syllabus topics and status.
- University Syllabus: show a programme-level overview and allow a syllabus source/reference field.
- Calendar: show academic and leave events filtered by academic year and semester.
- Assessments: show IA1, IA2, assignments, lab evaluations, project reviews and exams.
- Status Tracker: use Not Started / In Progress / Completed and show completion counts.
- Scorecard: calculate semester percentage from scores.csv and show grades. Do not invent a university CGPA formula; label any simple average clearly as a prototype metric.
- Plan & Checklist: create editable checklist items with category, target date, status and notes. Store the checklist in data/plan_checklist.csv.
- Ask Degree AI: create the interface now. If GEMINI_API_KEY is unavailable, answer common questions from the loaded CSV data using simple local logic and show "AI API not connected" for questions that need generation.
- Data Admin: use st.data_editor so the student can edit data tables and save the updated CSV files.

VISUAL DESIGN
- professional college dashboard
- clear headings and subheadings
- use cards / metrics sparingly
- no clutter
- no fake data beyond the CSV files

Return ONLY complete app.py code first.
```

Phase 4 - Build the academic modules | 45-70 min

> **Output for this phase**
> Semesters, courses, syllabus, calendar and assessments all render from CSV data.

1. Open Semesters and verify all eight semester labels are visible.
2. Open Courses & Syllabus. Select Semester 1, then Semester 3. The table must change based on the selected semester.
3. Open University Syllabus. Add a short programme-level summary such as programme name, total semesters, expected credits, and a syllabus source/reference field. Do not invent regulations.
4. Open Academic / Leave Calendar. Filter by academic year and semester. Confirm holidays and academic events can coexist in one table.
5. Open Internal Assessments. Confirm IA, assignment, lab/project review and exam records are all supported.

6. Change one CSV row manually, refresh the app, and confirm the screen updates. This proves the data is maintainable.

Phase 5 - Build status, scorecard, and the Plan & Checklist | 70-85 min

> **Output for this phase**
> A management view that tells the student what is complete, what is pending, and what to do next.

1. On Status Tracker, count courses by Not Started / In Progress / Completed and show a progress bar for the selected semester.

2. On Scorecard, calculate simple percentage = total score / total max marks × 100 for the available sample data. Clearly label it “prototype score percentage”, not an official university CGPA unless you have the actual formula.

3. Create data/plan_checklist.csv with columns: category, item, target_date, status, notes.

4. Add checklist categories: Semester Preparation, Assessments, Labs, Projects, Documents, Skills, Internship/Placement.

5. Use st.data_editor on Plan & Checklist so the student can change status from Not Started → In Progress → Done.

6. Add summary metrics: total checklist items, done, due soon, overdue.

```
plan_checklist.csv
category,item,target_date,status,notes
Semester Preparation,Download current syllabus,2026-11-03,Done,
Assessments,Prepare for IA1 - Data Structures,2026-11-09,In Progress,Revise trees and graphs
Projects,Choose mini-project problem statement,2026-11-12,Not Started,
Documents,Update resume and marksheets folder,2026-12-01,Not Started,
Placement,Complete one aptitude mock test,2026-11-15,Not Started,
```

Phase 6 - Add Ask Degree AI | 85-105 min

> **Output for this phase**
> A chatbot that answers from the degree data and can optionally use Gemini for natural-language generation.

1. Open Google AI Studio and create an API key from the free tier if your instructor allows external AI API use.

2. Never paste the API key directly into app.py. For local testing, place it in .streamlit/secrets.toml as GEMINI_API_KEY="...".

3. Paste the Chatbot Upgrade Prompt below into your coding assistant and ask it to modify ONLY the Ask Degree AI section.

4. Give the chatbot a compact context built from the loaded CSVs. Tell it to use that context as the source of truth and say it cannot verify information that is not present.

5. Ask these test questions: “What courses are still incomplete in Semester 3?”, “When is my next internal assessment?”, “Show my next holiday”, “What topics are in Data Structures?”, and “What is my official university CGPA formula?”.

6. The last question must trigger a safe fallback unless the official formula is actually present in your data.

Chatbot Upgrade Prompt

```
Modify the Ask Degree AI section of my Streamlit app.
```

Requirements:
1. Build a context string only from semesters.csv, courses.csv, academic_calendar.csv, assessments.csv, scores.csv and plan_checklist.csv.
2. If GEMINI_API_KEY exists in st.secrets, use google-genai with a free-tier model available to the account.
3. System instruction: "You are Degree AI. Answer only from the supplied degree data. Do not invent university rules, dates, marks, formulas or syllabus content. If the requested information is missing, say: I cannot verify that from the current Degree OS data."
4. Keep answers concise and student-friendly.
5. Show which data area was used: Courses, Calendar, Assessments, Scores or Plan.
6. If there is no API key, support deterministic local answers for: incomplete courses, next assessment, next holiday, course syllabus topics, semester status and checklist status.
7. Preserve the rest of the app unchanged.

Return the revised code block for the Ask Degree AI section and any new imports required.

> **Safe-AI rule**
> The chatbot is not allowed to invent the official university CGPA formula, exam regulation, eligibility rule, or syllabus. If you have not loaded the source, the correct answer is “I cannot verify that from the current Degree OS data.”

Phase 7 - Test like an engineer | 105-112 min

> **Output for this phase**
> A short evidence log showing the product works and fails safely.

1. Run every navigation item once. No page may crash.
2. Change a course status in Data Admin, save it, and confirm Status Tracker changes.
3. Add one new holiday in Data Admin and confirm it appears in the Calendar.
4. Add one new score and confirm the Scorecard recalculates.
5. Mark one Plan & Checklist item Done and confirm the checklist summary changes.
6. Ask the chatbot one question that is present in the data and one question that is absent. Record both responses.

Seven acceptance tests

- ☐ All 11 required navigation items are visible.
- ☐ Semester 1 to Semester 8 can all be selected.
- ☐ Course data changes when the semester filter changes.
- ☐ Calendar can distinguish academic events and leave/holidays.
- ☐ Assessment dates and score data are visible.
- ☐ Plan & Checklist can be edited without touching code.
- ☐ Chatbot gives a grounded answer and a safe “cannot verify” response.

Phase 8 - Launch the working app | 112-120 min

> **Output for this phase**
> A public Streamlit URL plus a GitHub repository containing the code and data.

1. Create or open your GitHub repository.
2. Upload app.py, requirements.txt and the data folder. Do NOT upload secrets.toml or API keys.
3. Open Streamlit Community Cloud and sign in with GitHub.
4. Choose Create app → select your repository → branch → app.py → Deploy.
5. If using Gemini, open the deployed app settings / Secrets and add GEMINI_API_KEY there rather than committing it to GitHub.
6. Wait for the deployment to finish, open the generated streamlit.app URL, and run one quick chatbot test.
7. Copy the app URL and repository URL into the submission section.

4. Required design for the Plan & Checklist tab

This is not a decorative to-do list. It is the student's degree execution control panel.

Section	What it must show	Example
This Week	Tasks due in the next 7 days	Prepare IA1; submit lab record
Semester Milestones	Major outcomes for current semester	Complete mini project; earn 24 credits
Degree Milestones	Long-range checkpoints	Internship; capstone; placement prep
Status	Not Started / In Progress / Done	Editable through st.data_editor
Health	Done / Due Soon / Overdue	Calculated from target date
Notes	Short execution notes	"Revise unit 3 before Friday"

5. Final launch checklist

☐ App opens without an error.
☐ Dashboard shows meaningful degree metrics.
☐ All 8 semesters exist.
☐ Courses and syllabus are filterable by semester.
☐ University Syllabus page includes a source/reference field.
☐ Academic / Leave Calendar works by year/semester.
☐ Internal assessments are visible and date-based.
☐ Status Tracker shows Not Started / In Progress / Completed.
☐ Scorecard calculates from the loaded score data.
☐ Plan & Checklist is editable and tracks completion.
☐ Chatbot answers at least 5 supported degree questions.
☐ Chatbot refuses one unsupported university-rule question.
☐ Data Admin can edit at least one table and save it.
☐ GitHub repo is updated.
☐ Streamlit app URL works in a new browser window.

6. What to submit

Team / Student	

GitHub repository URL	
Live Streamlit app URL	
One screenshot	Dashboard + chatbot visible
One failure/safety example	Question the chatbot correctly refused
One improvement for next version	

7. Instructor validation rubric (100 points)

Area	Marks	Validation
Navigation + usability	10	Required tabs/pages visible; no confusing navigation.
8-semester model	10	Semesters 1-8 represented consistently.
Courses + syllabus	10	Filterable data and clear syllabus/topic field.
Calendar + assessments	10	Academic/leave dates and internal assessments work.
Status + scorecard	10	Progress/status and score calculations are data-driven.
Plan & Checklist	15	Editable, due-date aware, and useful for execution.
Data maintainability	10	CSV/Data Admin update works without code changes.
Chatbot grounding	15	Answers from data; refuses unsupported rules/facts.
Launch quality	10	Live URL works and repository is complete.

8. Troubleshooting - solve these without instructor help

Problem	Fix
streamlit: command not found	Run: python -m pip install streamlit, then: python -m streamlit run app.py
FileNotFoundError for CSV	Check the data/ folder name and exact CSV filename.
Column not found	Compare your CSV headers with the column names used in app.py.
App deploys but chatbot fails	Add GEMINI_API_KEY in Streamlit app Secrets; never commit the key.
Gemini rate limit / unavailable	Use the local fallback chatbot. The assignment still passes if grounded local Q&A works.
Score looks wrong	Check max_marks and missing values. Do not silently fill official CGPA rules.
Data Admin does not persist	Confirm the code writes the edited DataFrame back to the same CSV file.

Too much to finish	Stop styling. Make every required page work with sample data first.

9. If you finish early: Version 2 ideas

- ☐ Upload the real university syllabus PDF and extract course metadata.
- ☐ Add per-semester credit requirements and official CGPA formula after verifying university rules.
- ☐ Add attendance tracking.
- ☐ Add reminders / notifications.
- ☐ Add document vault for marksheets, certificates and project files.
- ☐ Add personalised placement readiness checklist.
- ☐ Add login and multi-student accounts.
- ☐ Replace CSV storage with SQLite / PostgreSQL for multi-user deployment.

10. Free-tool references verified for this assignment

Streamlit Community Cloud: Free community deployment and GitHub-connected workflow. Source

Streamlit deployment docs: Community Cloud is intended for free community / educational deployment. Source

GitHub pricing: GitHub Free includes unlimited public/private repositories. Source

Gemini API pricing: Google AI Studio access and free-tier API usage are available for supported models/limits. Source

> **Tool availability note**
> Free tiers and limits can change. The assignment is designed so the core app still works without the Gemini API: the local data, dashboard, tracking, checklist and deterministic Q&A remain fully functional.